

FSTREAM_REFERENCE

Core Definition

`<fstream>` provides file-based stream classes: `ifstream` (read), `ofstream` (write), and `fstream` (read/write), plus the underlying `filebuf`. They inherit the full `istream/ostream` interface, so every formatting flag, manipulator and state function from `<iostream>` applies unchanged — files just swap in a file-backed stream buffer.

#CPP #FSTREAM #FILES #IO #IFSTREAM #OFSTREAM #BINARY-IO #FILE-HANDLING

CLASS HIERARCHY

```
basic_ios<CharT>
├── basic_istream — basic_ifstream<CharT>    ifstream / wifstream
├── basic_ostream — basic_ofstream<CharT>    ofstream / wofstream
└── basic_iostream — basic_fstream<CharT>    fstream / wfstream

basic_streambuf — basic_filebuf<CharT>      filebuf / wfilebuf
```

COMPLETE FSTREAM QUICK REFERENCE

CONSTRUCTION, OPENING & CLOSING — Target | Operation | Output

```
// =====
// CONSTRUCTION
// =====
std::ifstream f; // none | Default, unopened | Call open() later
std::ifstream f(name); // path/string | Open for reading | Sets failbit if open fails
std::ifstream f(name, mode); // +openmode | Open with mode | mode OR'd with ios::in
std::ofstream f(name); // path/string | Open for writing | Truncates by default
std::ofstream f(name, mode); // +openmode | Open with mode | mode OR'd with ios::out
std::fstream f(name, mode); // +openmode | Open read/write | NO implicit mode – specify both
std::ifstream f(std::move(other)); // rvalue | Move construct | C++11; streams are movable, not
    copyable

// Name accepts: const char*, std::string (C++11),
// std::filesystem::path (C++17)

// =====
// OPEN / CLOSE / STATUS
// =====
f.open(name) // path | Open with default mode | Returns void; sets failbit on
    error
f.open(name, mode) // +openmode | Open with mode | Returns void
f.is_open() // none | Check if file is open | Returns bool
f.close() // none | Flush and close | Returns void; sets failbit if
    not open
f.rdbuf() // none | Get the filebuf | Returns basic_filebuf*
f = std::move(other); // rvalue | Move assign | C++11

// =====
// OPEN MODE FLAGS (std::ios_base::openmode) – combine with |
// =====
std::ios::in // Open for reading
std::ios::out // Open for writing
std::ios::app // Append: ALL writes go to end, seekp cannot override
```

```

std::ios::ate           // "At end": seek to end once on open, then free to seek
std::ios::trunc         // Discard existing contents on open
std::ios::binary        // No newline / EOF translation (matters on Windows)
std::ios::noreplace     // C++23: fail if the file already exists (exclusive create)

// Class defaults (implicitly OR'd in):
//   ifstream → ios::in
//   ofstream → ios::out | ios::trunc
//   fstream  → NONE (you must supply the mode)

// =====
// COMMON MODE COMBINATIONS
// =====
std::ios::in           // Read text (ifstream default)
std::ios::out           // Write text, truncate (ofstream default)
std::ios::out | std::ios::app // Append text, create if missing
std::ios::in | std::ios::out // Read/write existing file; fails if absent
std::ios::in | std::ios::out | std::ios::trunc // Read/write, wipe/create
std::ios::in | std::ios::out | std::ios::ate // Read/write existing, start at end
std::ios::in | std::ios::binary // Read binary
std::ios::out | std::ios::binary // Write binary
std::ios::in | std::ios::out | std::ios::binary // Random-access binary update

// =====
// READING (inherited from istream)
// =====
in >> value           // Formatted extraction | Whitespace-delimited tokens
std::getline(in, str) // Line into std::string | Discards '\n'
std::getline(in, str, delim) // Custom delimiter | e.g. ',' for CSV fields
in.get()              // One char | Returns int_type / EOF
in.get(ch)            // One char into ch | Returns istream&
in.get(buf, n[, delim]) // C-string, LEAVES delim | Null-terminates
in.getline(buf, n[, delim]) // C-string, EATS delim | Null-terminates
in.read(buf, count) // Raw block | Binary reads; check gcount()
in.readsome(buf, count) // Available bytes only | Returns streamsize
in.gcount()           // Chars from last unformatted read | Returns streamsize
in.peek()             // Next char, no consume | Returns int_type
in.ignore(n[, delim]) // Discard characters | Returns istream&
in.putback(ch) / in.unget() // Push back | Returns istream&

// =====
// WRITING (inherited from ostream)
// =====
out << value           // Formatted insertion | All manipulators apply
out.put(ch)            // One char | Returns ostream&
out.write(buf, count) // Raw block | Binary writes
out.flush()            // Force buffer to disk | Returns ostream&

// =====
// FILE POSITIONING
// =====
in.tellg()             // none | Current read position | Returns pos_type (-1 on fail)
in.seekg(pos)          // pos_type | Absolute read seek | Returns istream&
in.seekg(off, dir)     // off+dir | Relative read seek | dir: ios::beg/cur/end
out.tellp()            // none | Current write position | Returns pos_type
out.seekp(pos)         // pos_type | Absolute write seek | Returns ostream&
out.seekp(off, dir)    // off+dir | Relative write seek | dir: ios::beg/cur/end

std::ios::beg          // Offset from start
std::ios::cur          // Offset from current position
std::ios::end          // Offset from end (use negative offsets)

// =====
// STREAM STATE (inherited from basic_ios) – see IOSTREAM_REFERENCE
// =====
f.good() / f.eof() / f.fail() / f.bad() // State queries | Returns bool
f.rdstate() / f.clear() / f.setstate() // State manipulation
f.exceptions(mask) // Throw on flags | ios_base::failure
static_cast<bool>(f) // !fail() | `if (f)` / `while (getline(f,s))`

// =====
// FILEBUF (low level)

```

//			
std::filebuf fb;	// none	Raw file buffer	Usable with any stream
fb.open(name, mode)	// path+mode	Open	Returns filebuf* or nullptr
fb.close()	// none	Close	Returns filebuf* or nullptr
fb.is_open()	// none	Check open	Returns bool
fb.pubseekoff(off, dir[, which])	// Seek relative		Returns pos_type
fb.pubseekpos(pos[, which])	// Seek absolute		Returns pos_type
fb.pubsetbuf(buf, n)	// Supply your own buffer		Returns streambuf*
fb.in_avail()	// Bytes available without blocking		Returns streamsize
fb.sgetc() / fb.sbumpc() / fb.sputc(c)	// Character-level buffer access		
std::ostream os(&fb);	// Attach a stream to a filebuf		

COMMON PATTERNS & EXAMPLES

Read a File Line by Line

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ifstream in("data.txt");
    if (!in) {
        std::cerr << "Cannot open data.txt\n";
        return 1;
    }

    std::string line;
    int lineNo = 0;
    while (std::getline(in, line)) {
        std::cout << ++lineNo << ": " << line << '\n';
    }

    if (in.bad()) std::cerr << "I/O error while reading\n";
    // Destructor closes the file – no explicit close() needed
}
```

Write a File

```
#include <fstream>
#include <iomanip>

int main() {
    std::ofstream out("report.txt");
    if (!out) return 1;

    out << "Report\n" << std::string(20, '=') << '\n';
    out << std::fixed << std::setprecision(2);
    out << "Total: " << 1234.5 << '\n';
} // Flushed and closed by the destructor
```

Append Instead of Overwrite

```
std::ofstream log("app.log", std::ios::app);
log << "[event] started\n";
// With ios::app every write goes to the end, even after seekp.
```

Read an Entire File into a String

```
#include <fstream>
#include <sstream>
#include <string>

std::string slurp(const std::string& path) {
    std::ifstream in(path, std::ios::binary);
    if (!in) throw std::runtime_error("cannot open " + path);
    std::ostringstream ss;
    ss << in.rdbuf();           // Buffer-to-buffer copy – fast, no loop
    return ss.str();
}

// Size-based alternative (avoids the extra copy):
std::string slurp2(const std::string& path) {
    std::ifstream in(path, std::ios::binary | std::ios::ate);
    if (!in) throw std::runtime_error("cannot open " + path);
    auto size = in.tellg();
    std::string buf(static_cast<std::size_t>(size), '\0');
    in.seekg(0);
    in.read(buf.data(), size); // C++17: data() is non-const
    buf.resize(static_cast<std::size_t>(in.gcount()));
    return buf;
}
```

Copy a File

```
#include <fstream>

bool copyFile(const std::string& src, const std::string& dst) {
    std::ifstream in(src, std::ios::binary);
    std::ofstream out(dst, std::ios::binary);
    if (!in || !out) return false;
    out << in.rdbuf();           // Whole-buffer copy
    return out.good();
}

// C++17 alternative: std::filesystem::copy_file(src, dst);
```

Binary I/O with POD Structs

```
#include <fstream>
#include <vector>
#include <type_traits>

struct Record {
    int    id;
    double value;
    char   name[32];
};
static_assert(std::is_trivially_copyable_v<Record>);

void writeRecords(const std::vector<Record>& recs, const char* path) {
    std::ofstream out(path, std::ios::binary);
    out.write(reinterpret_cast<const char*>(recs.data()),
              static_cast<std::streamsize>(recs.size() * sizeof(Record)));
}

std::vector<Record> readRecords(const char* path) {
    std::ifstream in(path, std::ios::binary | std::ios::ate);
    if (!in) return {};
    auto bytes = in.tellg();
    in.seekg(0);
    std::vector<Record> recs(static_cast<std::size_t>(bytes) / sizeof(Record));
    in.read(reinterpret_cast<char*>(recs.data()), bytes);
    return recs;
}
```

```
}

```

Binary layouts are **not portable** across compilers, architectures or padding rules. For files that cross machines, serialize field by field with a fixed endianness and explicit sizes.

Random Access — Update a Record In Place

```
#include <fstream>

void updateRecord(const char* path, std::size_t index, const Record& r) {
    std::fstream f(path, std::ios::in | std::ios::out | std::ios::binary);
    if (!f) return;
    f.seekp(static_cast<std::streamoff>(index * sizeof(Record)), std::ios::beg);
    f.write(reinterpret_cast<const char*>(&r), sizeof(Record));
}
```

File Size

```
std::ifstream in(path, std::ios::binary | std::ios::ate);
std::streamsize size = in.tellg(); // ate → already at end
in.seekg(0, std::ios::beg);       // Rewind before reading

// C++17 preferred:
// auto size = std::filesystem::file_size(path);
```

Parsing CSV

```
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

std::vector<std::vector<std::string>> readCSV(const std::string& path) {
    std::vector<std::vector<std::string>> rows;
    std::ifstream in(path);
    std::string line;

    while (std::getline(in, line)) {
        std::vector<std::string> fields;
        std::istringstream ls(line);
        std::string field;
        while (std::getline(ls, field, ',')) {
            fields.push_back(field);
        }
        rows.push_back(std::move(fields));
    }
    return rows;
}

// Note: this does NOT handle quoted fields containing commas or newlines.
```

Reading Numbers Until EOF

```
std::ifstream in("numbers.txt");
double x, sum = 0;
int n = 0;
while (in >> x) { sum += x; ++n; }

if (!in.eof()) std::cerr << "Stopped early: malformed data\n";
std::cout << "Mean: " << (n ? sum / n : 0) << '\n';
```

Reusing One Stream Object

```
std::ifstream in;
for (const auto& path : paths) {
    in.open(path);
    if (!in) { in.clear(); continue; } // clear() BEFORE the next open
    // ... read ...
    in.close();
    in.clear();                       // Reset eofbit for the next iteration
}
```

Exception-Based Handling

```
#include <fstream>
#include <iostream>

int main() {
    std::ifstream in;
    in.exceptions(std::ios::failbit | std::ios::badbit); // NOT eofbit
    try {
        in.open("config.txt");
        std::string key; int value;
        while (in >> key >> value) { /* ... */ }
    } catch (const std::ios_base::failure& e) {
        std::cerr << "Failed: " << e.what() << '\n';
    }
}
```

Enabling eofbit in the exception mask makes normal end-of-file throw — almost never what you want.

Temporary Redirection of `cout` to a File

```
std::ofstream file("out.txt");
auto* old = std::cout.rdbuf(file.rdbuf());
std::cout << "captured\n";
std::cout.rdbuf(old); // Restore — mandatory
```

`std::filesystem` Companions (C++17)

```
#include <filesystem>
namespace fs = std::filesystem;

fs::exists(p); // bool
fs::file_size(p); // uintmax_t
fs::remove(p); // bool
fs::rename(from, to); // void
fs::copy_file(from, to); // bool
fs::create_directories(p); // bool
fs::last_write_time(p); // file_time_type
fs::is_regular_file(p); // bool
for (const auto& e : fs::directory_iterator(dir)) { /* e.path() */ }
```

IMPORTANT CONCEPTS

RAII: Files Close Themselves

The destructor closes and flushes. Explicit `close()` is only needed when you want to release the handle early, check for close-time errors, or reuse the stream object.

Always Check After Opening

```
std::ifstream in("maybe.txt");
if (!in) { /* handle */ } // Preferred
if (!in.is_open()) { /* ... */ } // Equivalent for the open case
```

Constructing an `ifstream` on a missing file does not throw by default — it sets `failbit` silently.

app VS ate

- `ios::app` — **every** write is forced to the end; `seekp` cannot redirect it.
- `ios::ate` — seek to the end once at open; you may seek anywhere afterward.

fstream Has No Default Mode

`std::fstream f("x.txt");` opens `in|out` and **fails if the file does not exist**. To create it: add `ios::trunc`, or open with `ofstream` first.

Switching Direction on an fstream

Between a read and a write (or vice versa) on the same `fstream`, you must intervene with a seek or a flush. Otherwise the behavior is undefined:

```
f >> value;
f.seekp(f.tellg()); // Reposition before writing
f << newValue;
```

Binary Mode Matters on Windows

Without `ios::binary`, `\n` is translated to `\r\n` on write and back on read, and a `0x1A` byte can end a text-mode read. Any non-text file must be opened with `ios::binary`.

Text-Mode `tellg()` Is Opaque

In text mode the value returned by `tellg()` is not necessarily a byte offset. Only use it as an argument to `seekg()`, never for arithmetic. In binary mode it is a byte offset.

`clear()` Before Reopening

`eofbit` persists after a read loop finishes. Reopening a stream that still has flags set gives a stream that fails immediately. Call `clear()` between uses.

Flush Only When It Matters

Data lives in the buffer until it is full, the stream is closed, or you flush. For crash-critical logging, use `std::endl`, `out.flush()`, or set `unitbuf`. In throughput-oriented code, let the buffer do its job.

Stream Sizes and Types

Use `std::streamsize` for counts, `std::streamoff` for offsets and `std::streampos` for positions. Casting between them and `std::size_t` warrants an explicit cast.

OPEN MODE MATRIX

Mode combination	File missing	Existing content	Position on open
in	fail	preserved	beginning
out	created	TRUNCATED	beginning
out trunc	created	TRUNCATED	beginning
out app	created	preserved	end (writes forced to end)
out in	fail	preserved	beginning
out in trunc	created	TRUNCATED	beginning
out in ate	fail	preserved	end
out noreplace (C++23)	created	fail	beginning
any binary	(as above)	(as above)	no newline translation

BEST PRACTICES

1. Check the stream after opening — `if (!in) { ... }`
2. Let RAII close the file; call `close()` only for early release or reuse
3. Use `ios::binary` for every non-text file — and for anything you read/write raw
4. Prefer `std::getline` over `in.getline(buf, n)` — no fixed buffer to overflow
5. Loop on the read, not on `eof()` — `while (std::getline(in, line))`
6. `clear()` before reopening a reused stream object
7. Seek or flush when switching read ↔ write on an `fstream`
8. Don't do arithmetic on text-mode `tellg()` values
9. Prefer `std::filesystem` for existence, size, rename, delete and directory work
10. Use `os << in.rdbuf()` to copy whole files — it beats a character loop
11. Don't dump structs to disk for portable formats — serialize explicitly
12. Keep `eofbit` out of the exception mask
13. Use `std::filesystem::path` for filenames (C++17) — handles Unicode paths correctly
14. Write to a temp file then rename for atomic updates of important files

RELATED HEADERS

```
#include <fstream>      // ifstream, ofstream, fstream, filebuf
#include <iostream>      // cin/cout/cerr + the stream interface
#include <sstream>       // in-memory streams (parsing lines, building strings)
#include <filesystem>    // C++17: paths, existence, size, copy, directories
#include <iomanip>       // formatting manipulators
#include <cstdio>        // C stdio: fopen/fread/remove/rename
#include <span>          // C++20: view over raw buffers for read/write
```

EXTERNAL RESOURCES

- `<fstream>` : <https://en.cppreference.com/w/cpp/header/fstream>
- `basic_fstream` : https://en.cppreference.com/w/cpp/io/basic_fstream
- `openmode` : https://en.cppreference.com/w/cpp/io/ios_base/openmode
- `<filesystem>` : <https://en.cppreference.com/w/cpp/filesystem>

Standard: C++11 (string paths, move) / C++17 (`filesystem::path`) / C++23 (`noreplace`) **Last Updated:** September 2026